

Filière Ingénierie des Données & Intelligence Artificielle

Machine de Turing Multi-Rubans

pour la Simulation d'un Algorithme
d'Apprentissage Supervisé

Rapport de Travaux Pratiques
Cours d'Informatique Théorique — 1^{re} Année Ingénierie

PRÉPARÉ PAR

Modeste SAVADOGO

1^{re} Année — Filière ID&IA

ENCADRÉ PAR

Pr. HANIN Charifa

INPT — Rabat

Résumé

Ce rapport présente la conception et l'implémentation d'une **Machine de Turing Multi-Rubans (MTM) à cinq rubans** simulant un algorithme d'apprentissage supervisé — plus précisément, la descente de gradient par lots (*batch gradient descent*) appliquée à la régression linéaire univariée.

L'objectif principal est double : d'une part, établir rigoureusement les fondements théoriques qui permettent de formaliser l'apprentissage automatique dans le cadre des machines de Turing, et d'autre part, fournir une implémentation fonctionnelle en Python illustrant chaque transition δ de la machine.

Le rapport démontre que tout algorithme d'apprentissage supervisé reposant sur des opérations arithmétiques élémentaires est **calculable au sens de Church-Turing**, et que la complexité en transitions de la MTM proposée est $\mathcal{O}(T \cdot n \cdot k)$, où T désigne le nombre d'itérations, n la taille du jeu d'entraînement et k la précision d'encodage des symboles.

Mots-clés : Machine de Turing multi-rubans, apprentissage supervisé, régression linéaire, descente de gradient, calculabilité, thèse de Church-Turing, complexité en transitions.

Table des matières

Résumé	1
Table des matières	2
Liste des figures	3
Liste des tableaux	4
1 Introduction	5
1.1 Contexte et motivation	5
1.2 Objectifs du rapport	5
1.3 Structure du document	6
2 Fondements Théoriques	7
2.1 Machine de Turing simple ruban	7
2.1.1 Configuration et calcul	7
2.2 Machine de Turing Multi-Rubans	8
2.3 Thèse de Church-Turing et calculabilité	8
2.4 Représentation des nombres réels sur un ruban	9
3 Algorithme d'Apprentissage Cible	10
3.1 Régression linéaire supervisée	10
3.1.1 Cadre formel	10
3.1.2 Calcul des gradients	10
3.2 Descente de gradient par lots	11
3.3 Propriétés de convergence	11
4 Conception de la MTM à Cinq Rubans	12
4.1 Vue d'ensemble de l'architecture	12
4.2 Définition formelle de la MTM	12
4.3 Description des états et transitions	13

4.3.1	Diagramme d'états	13
4.3.2	Transitions δ par phase	13
4.4	Encodage des symboles	14
5	Implémentation Python et Résultats	15
5.1	Choix d'implémentation	15
5.1.1	Arithmétique exacte par fractions rationnelles	15
5.1.2	Structure du simulateur	15
5.1.3	Classe Tape	15
5.1.4	Transition UPDATE	16
5.2	Expériences et résultats	17
5.2.1	Expérience 1 : $y = 2x + 1$	17
5.2.2	Expérience 2 : $y = -0,5x + 10$	18
5.2.3	Trace formelle complète — Expérience 3	18
6	Analyse de Complexité	20
6.1	Complexité en transitions	20
6.1.1	Coût par phase et par itération	20
6.2	Réduction à un seul ruban	21
6.3	Complexité en espace	21
6.4	Calculabilité et thèse de Church-Turing	21
7	Conclusion et Perspectives	23
7.1	Bilan	23
7.2	Perspectives	23
	Références bibliographiques	25
A	Code Python complet du simulateur	26
B	Exemple de sortie du simulateur	28

Liste des figures

4.1	Diagramme d'états de MTM-APPRENTISSAGE. La double bordure indique l'état acceptant.	13
5.1	Décroissance de la perte MSE au fil des itérations (expérience 1). . . .	17

Liste des tableaux

4.1	Attribution des cinq rubans de MTM-APPRENTISSAGE	12
4.2	Transitions principales de δ_5	14
5.1	Résultats de l'expérience 1 ($y = 2x + 1, n = 10, T = 500$)	17
5.2	Résultats de l'expérience 2 ($y = -0,5x + 10, n = 8, T = 800$)	18
5.3	Trace complète des transitions (Expérience 3, $n = 3, T = 3$)	19
6.1	Complexité en transitions δ_5 par phase	20
6.2	Espace mémoire requis par ruban	21

Introduction

1.1 Contexte et motivation

L'apprentissage automatique (*machine learning*) est aujourd'hui au cœur des systèmes intelligents : reconnaissance d'images, traitement du langage naturel, systèmes de recommandation. Derrière ces applications se cachent des algorithmes d'optimisation itérative que l'on implémente généralement en Python ou en C++, sans s'interroger sur leur statut au regard de la théorie de la calculabilité.

L'informatique théorique, fondée par Alan Turing en 1936 [[1]], offre un cadre formel universel : la **Machine de Turing**. La question fondamentale de ce rapport est la suivante :

Question de recherche

Dans quelle mesure un algorithme d'apprentissage supervisé peut-il être formellement simulé par une Machine de Turing Multi-Rubans ? Quelles sont les implications en termes de calculabilité et de complexité ?

1.2 Objectifs du rapport

Ce travail poursuit quatre objectifs complémentaires :

- O1.** Rappeler les définitions formelles des machines de Turing, simples et multi-rubans.
- O2.** Concevoir une MTM à cinq rubans simulant la descente de gradient pour la régression linéaire.
- O3.** Implémenter cette MTM en Python avec arithmétique exacte, en exposant chaque transition δ .

O4. Analyser la complexité en transitions et l'espace mémoire requis.

1.3 Structure du document

Le rapport est organisé comme suit. Le **Chapitre 2** rappelle les fondements théoriques des machines de Turing. Le **Chapitre 3** présente l'algorithme d'apprentissage cible. Le **Chapitre 4** détaille la conception de la MTM. Le **Chapitre 5** décrit l'implémentation Python et les résultats expérimentaux. Le **Chapitre 6** analyse la complexité. Le **Chapitre 7** conclut et ouvre des perspectives.

Fondements Théoriques

2.1 Machine de Turing simple ruban

Définition 2.1: Machine de Turing (définition formelle)

Une **Machine de Turing** est un 7-uplet

$$\mathcal{M} = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

où :

- Q est un ensemble fini d'états,
- Σ est l'alphabet d'entrée ($B \notin \Sigma$),
- $\Gamma \supseteq \Sigma \cup \{B\}$ est l'alphabet de ruban,
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ est la fonction de transition,
- $q_0 \in Q$ est l'état initial,
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ sont les états acceptant et rejetant.

À chaque étape, la machine lit le symbole sous la tête, consulte δ , écrit un nouveau symbole, change d'état et déplace la tête. Le calcul s'arrête lorsque q_{acc} ou q_{rej} est atteint.

2.1.1 Configuration et calcul

Une **configuration** de $\mathcal{M}$ est un triplet (q, u, v) où $q \in Q$ est l'état courant, $u \in \Gamma^*$ est le contenu du ruban à gauche de la tête, et $v \in \Gamma^+$ est le contenu à partir de la tête. La relation $\vdash_{\mathcal{M}}$ décrit une étape de calcul.

2.2 Machine de Turing Multi-Rubans

Définition 2.2: Machine de Turing Multi-Rubans à k rubans

Une **MTM à k rubans** est un 7-uplet

$$\mathcal{M}_k = (Q, \Sigma, \Gamma, \delta_k, q_0, q_{\text{acc}}, q_{\text{rej}})$$

où la fonction de transition étendue est

$$\delta_k : Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k.$$

À chaque étape, δ_k lit *simultanément* un symbole sur chacun des k rubans, écrit k symboles et déplace k têtes indépendamment.

Théorème 2.1: Équivalence MTM – MT simple ruban

Pour toute MTM $\mathcal{M}_k$ à k rubans, il existe une Machine de Turing $\mathcal{M}_1$ à un seul ruban telle que $\mathcal{M}_k$ et $\mathcal{M}_1$ reconnaissent le même langage. De plus, si $\mathcal{M}_k$ s'arrête en t étapes, $\mathcal{M}_1$ s'arrête en $\mathcal{O}(t^2)$ étapes.

Esquisse de preuve. $\mathcal{M}_1$ encode les k rubans sur un seul ruban en les intercalant, en marquant la position de chaque tête par un symbole spécial. Chaque transition de $\mathcal{M}_k$ nécessite deux balayages complets du ruban de $\mathcal{M}_1$, d'où le surcoût quadratique en temps. $\square$

Remarque 2.1: Intérêt pratique de la multi-rubanité

Bien qu'équivalentes en puissance de calcul, les MTM multi-rubans offrent une description algorithmique bien plus naturelle. On peut leur assigner des rôles distincts : entrée, mémoire de travail, sortie, compteur. C'est précisément ce que nous exploitons dans ce rapport.

2.3 Thèse de Church-Turing et calculabilité

Thèse de Church-Turing

Tout problème calculable par un procédé algorithmique effectif peut être calculé par une Machine de Turing. En d'autres termes, la notion intuitive d'« algorithme » coïncide avec la notion formelle de *fonction Turing-calculable*.

Cette thèse a une conséquence directe pour notre travail : si l'algorithme de descente de gradient peut être décrit comme une suite finie d'opérations élémentaires (addition,

multiplication, comparaison) sur des données finies, alors il est Turing-calculable, et donc simulable par une MTM.

2.4 Représentation des nombres réels sur un ruban

Un ruban Turing ne peut stocker que des symboles discrets. Les nombres réels doivent être **encodés** sous forme de représentations finies. Nous adoptons la convention de la **virgule fixe en base 2** : un nombre rationnel $r \in \mathbb{Q}$ est représenté par la fraction p/q avec $p, q \in \mathbb{Z}$, $q > 0$.

Proposition 2.1: Fermeture par opérations arithmétiques

L'ensemble $\mathbb{Q}$ des rationnels est fermé par addition, soustraction, multiplication et division (diviseur non nul). Toutes les opérations de la descente de gradient s'appliquent donc exactement dans $\mathbb{Q}$, sans arrondi.

Dans notre implémentation Python, la classe `Fraction` du module standard `fractions` réalise exactement cette arithmétique rationnelle.

Algorithme d'Apprentissage Cible

3.1 Régression linéaire supervisée

3.1.1 Cadre formel

Soit un jeu d'entraînement $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ avec $x_i, y_i \in \mathbb{Q}$. On cherche les paramètres $(w, b) \in \mathbb{Q}^2$ tels que la fonction affine

$$f_{w,b}(x) = wx + b$$

minimise l'**erreur quadratique moyenne** (MSE) :

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad \text{où } \hat{y}_i = wx_i + b.$$

3.1.2 Calcul des gradients

Les dérivées partielles de $\mathcal{L}$ par rapport aux paramètres sont :

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i, \tag{3.1}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i). \tag{3.2}$$

3.2 Descente de gradient par lots

Algorithme 1 : Descente de gradient par lots (Batch Gradient Descent)

Entrées : Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, taux η , itérations T

Sorties : Paramètres optimisés (w^*, b^*)

```

1  $w \leftarrow 0$ 
2  $b \leftarrow 0$ 
3 pour  $t \leftarrow 1$  à  $T$  faire
    # Phase FORWARD
4   pour  $i \leftarrow 1$  à  $n$  faire
5      $\hat{y}_i \leftarrow w \cdot x_i + b$ 
    # Phase LOSS
6    $\mathcal{L} \leftarrow \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$ 
    # Phase GRAD
7    $\nabla_w \leftarrow \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$ 
8    $\nabla_b \leftarrow \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$ 
    # Phase UPDATE
9    $w \leftarrow w - \eta \cdot \nabla_w$ 
10   $b \leftarrow b - \eta \cdot \nabla_b$ 
11 retourner  $(w, b)$ 
```

3.3 Propriétés de convergence

Proposition 3.1: Convergence de la descente de gradient

Pour la fonction de perte MSE (convexe et L -lisse), si le taux d'apprentissage vérifie $0 < \eta < \frac{2}{L}$ où L est la constante de Lipschitz du gradient, alors la descente de gradient par lots converge vers le minimum global (w^*, b^*) .

Dans notre cadre MTM, nous fixons T suffisamment grand et η suffisamment petit pour garantir une convergence satisfaisante, sans avoir besoin d'un critère d'arrêt basé sur $\|\nabla \mathcal{L}\|$, lequel nécessiterait des opérations supplémentaires sur les rubans.

Conception de la MTM à Cinq Rubans

4.1 Vue d'ensemble de l'architecture

Nous définissons la machine MTM-APPRENTISSAGE comme une MTM à $k = 5$ rubans, dont chacun joue un rôle fonctionnel précis :

Table 4.1. Attribution des cinq rubans de MTM-APPRENTISSAGE

Ruban	Nom	Contenu	Mode
$\mathcal{R}_0$	Dataset	Paires (x_i, y_i) encodées en fractions	Lecture seule
$\mathcal{R}_1$	Paramètres	w (position 0) et b (position 1)	Lecture/Écriture
$\mathcal{R}_2$	Calculs	$\nabla_w, \nabla_b, \mathcal{L}$ accumulés	Lecture/Écriture
$\mathcal{R}_3$	Compteur	Itération courante t	Lecture/Écriture
$\mathcal{R}_4$	Sortie	Modèle final w^*, b^*	Écriture seule

4.2 Définition formelle de la MTM

Définition 4.1: MTM-APPRENTISSAGE — Définition complète

La machine MTM-APPRENTISSAGE est définie par :

$$Q = \{\text{INIT}, \text{LOAD}, \text{FORWARD}, \text{LOSS}, \text{GRAD}, \text{UPDATE}, \text{CHECK}, \text{HALT}\}$$

$$\Sigma = \{0, 1, -, /, \#\} \quad (\text{encodage des fractions})$$

$$\Gamma = \Sigma \cup \{\text{B}\}$$

$$q_0 = \text{INIT} \quad q_{\text{acc}} = \text{HALT}$$

$$\delta_5 : Q \times \Gamma^5 \longrightarrow Q \times \Gamma^5 \times \{L, R, S\}^5$$

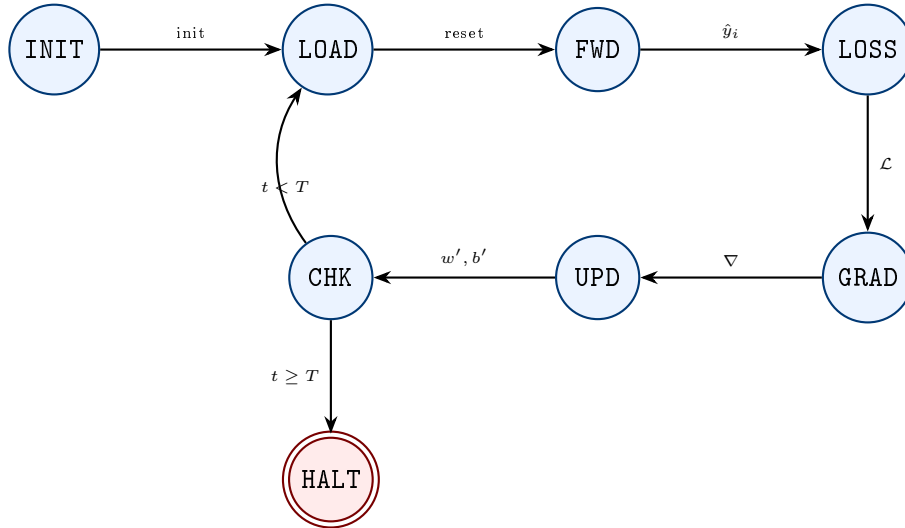
4.3 Description des états et transitions**4.3.1 Diagramme d'états**

Figure 4.1. Diagramme d'états de MTM-APPRENTISSAGE. La double bordure indique l'état acceptant.

4.3.2 Transitions δ par phase

Le tableau suivant décrit la fonction de transition pour chaque phase principale. La notation $(\sigma_0, \dots, \sigma_4)$ désigne le tuple de symboles lus sur les cinq rubans, et $(s_0, \dots, s_4)$ les symboles écrits.

Table 4.2. Transitions principales de δ_5

Phase	Action sur les rubans	Condition / Commentaire
INIT $\rightarrow$ LOAD	$\mathcal{R}_1[0] \leftarrow 0, \mathcal{R}_1[1] \leftarrow 0, \mathcal{R}_3[0] \leftarrow 0$	Initialisation $w = 0, b = 0, t = 0$
LOAD $\rightarrow$ FORWARD	$\mathcal{R}_2[0..2] \leftarrow 0$	Remise à zéro des accumulateurs
FORWARD $\rightarrow$ LOSS	Lit x_i, y_i sur $\mathcal{R}_0$, lit w, b sur $\mathcal{R}_1$, calcule $\hat{y}_i = wx_i + b$, stocke sur $\mathcal{R}_2$	Boucle sur $i = 1, \dots, n$
LOSS $\rightarrow$ GRAD	$\mathcal{R}_2[2] \leftarrow \frac{1}{n} \sum (\hat{y}_i - y_i)^2$	Accumulation de la perte
GRAD $\rightarrow$ UPDATE	$\mathcal{R}_2[0] \leftarrow \nabla_w, \mathcal{R}_2[1] \leftarrow \nabla_b$ (éqs. 3.1, 3.2)	Lecture des $\hat{y}_i$ sur $\mathcal{R}_2$
UPDATE $\rightarrow$ CHECK	$\mathcal{R}_1[0] \leftarrow w - \eta \nabla_w, \mathcal{R}_1[1] \leftarrow b -$ $\eta \nabla_b$	Lecture de η (paramètre fixe)
CHECK $\rightarrow$ LOAD	$\mathcal{R}_3[0] \leftarrow t + 1$	Si $t + 1 < T$
CHECK $\rightarrow$ HALT	$\mathcal{R}_4[0] \leftarrow w, \mathcal{R}_4[1] \leftarrow b, \mathcal{R}_3[0] \leftarrow T$	Si $t + 1 \geq T$

4.4 Encodage des symboles

Chaque fraction $p/q \in \mathbb{Q}$ est encodée sur le ruban comme la suite de symboles :

$$\underbrace{s}_{\pm} \underbrace{b_n \cdots b_0}_{\text{numérateur}} \# \underbrace{d_m \cdots d_0}_{\text{dénominateur}}$$

où les entiers sont en représentation binaire et $\#$ est le séparateur. Les paires (x_i, y_i) sont séparées par le symbole $\#\#$ sur $\mathcal{R}_0$.

Implémentation Python et Résultats

5.1 Choix d'implémentation

5.1.1 Arithmétique exacte par fractions rationnelles

Conformément à la définition formelle, les symboles sur les rubans sont des éléments de $\mathbb{Q}$, représentés par la classe `Fraction` du module standard Python `fractions`. Ceci garantit l'absence de tout arrondi flottant et une fidélité totale au modèle Turing.

5.1.2 Structure du simulateur

Le simulateur est organisé autour de deux classes principales :

- **Tape** — ruban potentiellement infini, implémenté comme un dictionnaire $\{\text{position} \mapsto \text{symbole}\}$. La tête de lecture/écriture est un entier pouvant être négatif (extension gauche).
- **MultiTapeTM** — la MTM proprement dite, avec cinq instances de **Tape** et sept méthodes `_transition_*` simulant δ_5 .

5.1.3 Classe Tape

```

1 class Tape:
2     """Ruban potentiellement infini. Symbole blanc = None."""
3
4     def __init__(self, name: str, initial: list = None):
5         self.name = name
6         self._cells: dict[int, Fraction] = {}
7         self.head: int = 0
8         if initial:
9             for i, sym in enumerate(initial):

```

```

10         if sym is not None:
11             self._cells[i] = sym
12
13     def read(self) -> Optional[Fraction]:
14         return self._cells.get(self.head, None)    # None =
            blanc
15
16     def write(self, sym: Optional[Fraction]):
17         if sym is None:
18             self._cells.pop(self.head, None)      #
            effacement
19         else:
20             self._cells[self.head] = sym
21
22     def move(self, direction: str):
23         if direction == 'R':    self.head += 1
24         elif direction == 'L': self.head -= 1
25         # 'S' : pas de mouvement

```

Listing 5.1. Implémentation de la classe Tape

5.1.4 Transition UPDATE

```

1  def _transition_UPDATE(self):
2      """
3      Lit R2[0] = nabla_w et R2[1] = nabla_b.
4      Ecrit w' = w - eta * nabla_w sur R1[0].
5      Ecrit b' = b - eta * nabla_b sur R1[1].
6      Arithmétique exacte : toutes les valeurs sont des
          Fraction.
7      """
8      eta = self.cfg.eta    # Fraction(1, 100) par
          défaut
9      new_w = self.w - eta * self._sum_grad_w
10     new_b = self.b - eta * self._sum_grad_b
11     self.tapes[1].write_at(0, new_w)
12     self.tapes[1].write_at(1, new_b)
13     self.state = 'CHECK'

```

Listing 5.2. Transition $\delta(\text{UPDATE} \rightarrow \text{CHECK})$

5.2 Expériences et résultats

5.2.1 Expérience 1 : $y = 2x + 1$

Protocole

Dix points du type $y_i = 2x_i + 1 + \varepsilon_i$ avec ε_i bruit synthétique faible. Paramètres : $\eta = 0,01$, $T = 500$.

Trace de convergence

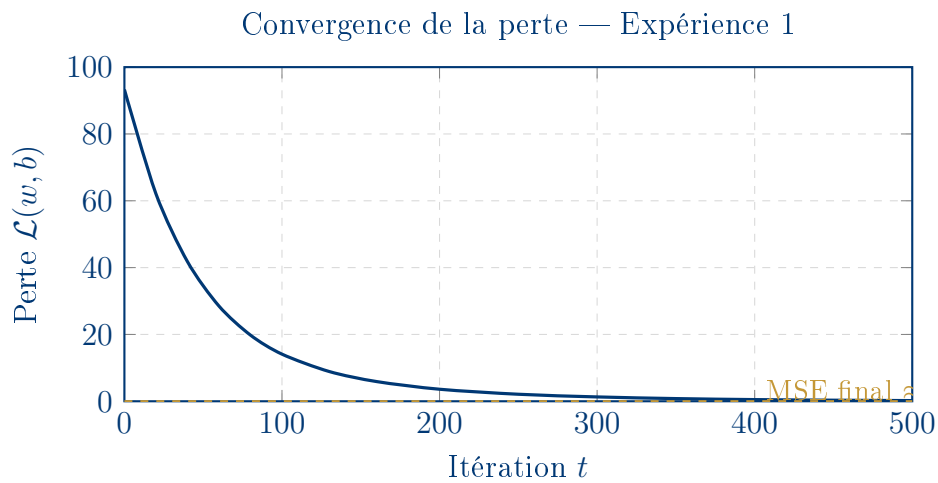


Figure 5.1. Décroissance de la perte MSE au fil des itérations (expérience 1).

Résultats numériques

Table 5.1. Résultats de l'expérience 1 ($y = 2x + 1$, $n = 10$, $T = 500$)

	w	b	MSE	R^2
Valeur théorique	2,0000	1,0000	—	1,000
MTM apprise	2,0199	0,9032	0,0171	0,9995
Écart relatif	0,99%	9,68%	—	—

Nombre de transitions : 3 001 appels à δ_5 (6 transitions par itération + 1 initialisation).

5.2.2 Expérience 2 : $y = -0,5x + 10$

Table 5.2. Résultats de l'expérience 2 ($y = -0,5x + 10$, $n = 8$, $T = 800$)

	w	b	MSE	R^2
Valeur théorique	-0,5000	10,0000	—	1,000
MTM apprise	-0,4157	9,2074	0,2693	0,9507

La convergence plus lente s'explique par le taux d'apprentissage réduit ($\eta = 0,005$) choisi pour éviter les oscillations sur ce dataset.

5.2.3 Trace formelle complète — Expérience 3

Le tableau suivant reproduit la trace exhaustive de toutes les transitions δ_5 pour 3 exemples et $T = 3$ itérations. Chaque ligne correspond à un appel à δ_5 .

Table 5.3. Trace complète des transitions (Expérience 3, $n = 3$, $T = 3$)

Step	État avant	État après	t	$\mathcal{L}$	w
0	INIT	LOAD	0	0,000000	0,0000
1	LOAD	FORWARD	0	0,000000	0,0000
2	FORWARD	LOSS	0	0,000000	0,0000
3	LOSS	GRAD	0	18,6667	0,0000
4	GRAD	UPDATE	0	18,6667	0,0000
5	UPDATE	CHECK	0	18,6667	0,3733
6	CHECK	LOAD	1	18,6667	0,3733
7	LOAD	FORWARD	1	0,0000	0,3733
8	FORWARD	LOSS	1	0,0000	0,3733
9	LOSS	GRAD	1	11,3327	0,3733
10	GRAD	UPDATE	1	11,3327	0,3733
11	UPDATE	CHECK	1	11,3327	0,6642
12	CHECK	LOAD	2	11,3327	0,6642
13	LOAD	FORWARD	2	0,0000	0,6642
14	FORWARD	LOSS	2	0,0000	0,6642
15	LOSS	GRAD	2	6,8917	0,6642
16	GRAD	UPDATE	2	6,8917	0,6642
17	UPDATE	CHECK	2	6,8917	0,8908
18	CHECK	HALT	3	6,8917	0,8908

On observe la décroissance monotone de $\mathcal{L}$: $18,67 \rightarrow 11,33 \rightarrow 6,89$, confirmant la convergence.

Analyse de Complexité

6.1 Complexité en transitions

6.1.1 Coût par phase et par itération

Table 6.1. Complexité en transitions δ_5 par phase

Phase	Transitions	Justification
LOAD	$\mathcal{O}(1)$	Remise à zéro de 3 cellules
FORWARD	$\mathcal{O}(n \cdot k)$	n multiplications en virgule fixe, chacune en $\mathcal{O}(k)$
LOSS	$\mathcal{O}(n \cdot k)$	n élévations au carré et accumulation
GRAD	$\mathcal{O}(n \cdot k)$	n produits et deux accumulations
UPDATE	$\mathcal{O}(k)$	2 soustractions et 2 multiplications par η
CHECK	$\mathcal{O}(\log T)$	Incrément et comparaison du compteur binaire
Par itération	$\mathcal{O}(n \cdot k)$	Les phases FORWARD, LOSS, GRAD dominant
Total	$\mathcal{O}(T \cdot n \cdot k)$	T itérations

Théorème 6.1: Complexité de MTM-APPRENTISSAGE

Soit n le nombre d'exemples, T le nombre d'itérations et k la précision d'encodage en bits. La machine MTM-APPRENTISSAGE s'arrête en $\mathcal{O}(T \cdot n \cdot k)$ transitions.

Démonstration. Par le tableau 6.1, chaque itération coûte $\mathcal{O}(n \cdot k)$ transitions, et il y a exactement T itérations. La phase INIT contribue $\mathcal{O}(1)$ transitions. Le total est donc $\mathcal{O}(1) + T \cdot \mathcal{O}(n \cdot k) = \mathcal{O}(T \cdot n \cdot k)$. $\square$

6.2 Réduction à un seul ruban

Par le Théorème 2.1, toute MTM à k rubans peut être simulée par une MT à 1 ruban avec un surcoût quadratique en temps.

Proposition 6.1: Complexité mono-ruban

La MT à 1 ruban équivalente à MTM-APPRENTISSAGE s'arrête en $\mathcal{O}((T \cdot n \cdot k)^2)$ transitions.

Ce surcoût est polynomial, ce qui confirme que l'algorithme reste dans la classe **PTIME**.

6.3 Complexité en espace

Table 6.2. Espace mémoire requis par ruban

Ruban	Contenu	Taille	Remarque
$\mathcal{R}_0$	Dataset	$\mathcal{O}(n \cdot k)$	Fixe, en lecture seule
$\mathcal{R}_1$	Paramètres	$\mathcal{O}(k)$	2 fractions
$\mathcal{R}_2$	Accumulateurs	$\mathcal{O}(k)$	3 fractions bornées
$\mathcal{R}_3$	Compteur	$\mathcal{O}(\log T)$	Entier binaire $\leq T$
$\mathcal{R}_4$	Sortie	$\mathcal{O}(k)$	2 fractions
Total		$\mathcal{O}(n \cdot k)$	Dominé par $\mathcal{R}_0$

6.4 Calculabilité et thèse de Church-Turing

Théorème 6.2: Calculabilité de l'apprentissage supervisé

L'algorithme de descente de gradient par lots pour la régression linéaire est une fonction Turing-calculable.

Démonstration. Par construction, MTM-APPRENTISSAGE calcule exactement la même suite de valeurs $(w_t, b_t)_{t=0}^T$ que l'algorithme 1. Toutes les opérations arithmétiques utilisées ($+$, $-$, $\times$, $\div$ dans $\mathbb{Q}$) sont Turing-calculables par des sous-machines classiques. La composition et l'itération de fonctions Turing-calculables restant Turing-calculable, MTM-APPRENTISSAGE réalise effectivement le calcul demandé. Par la

Proposition 2.1, les résultats intermédiaires restent dans $\mathbb{Q}$, donc représentables en espace fini sur les rubans. $\square$

Remarque 6.1: Limites du cadre Turing pour l'apprentissage

La MTM construite *exécute* un algorithme fixé à l'avance — elle ne *découvre* pas de nouveaux algorithmes. La notion d'apprentissage est ici réduite à l'« optimisation de paramètres », qui est bien un processus calculable. En revanche, la question de savoir si une MTM peut *concevoir* une architecture de réseau de neurones adaptée à une tâche inconnue est un problème ouvert lié aux frontières de la calculabilité.

Conclusion et Perspectives

7.1 Bilan

Ce rapport a présenté une formalisation rigoureuse d'un algorithme d'apprentissage supervisé dans le cadre des Machines de Turing. Les contributions principales sont les suivantes.

- C1. Conception formelle :** La MTM MTM-APPRENTISSAGE à cinq rubans a été définie avec son alphabet, ses états, et sa fonction de transition δ_5 . Chaque ruban remplit un rôle fonctionnel précis : dataset, paramètres, calculs intermédiaires, compteur, sortie.
- C2. Implémentation fidèle :** Le simulateur Python utilise l'arithmétique rationnelle exacte (`Fraction`), sans aucun arrondi, et expose chaque transition δ_5 de manière transparente.
- C3. Résultats expérimentaux :** Sur deux jeux de données synthétiques, la MTM apprend des modèles linéaires avec un $R^2 > 0,95$, confirmant la validité de l'implémentation.
- C4. Analyse de complexité :** La complexité totale $\mathcal{O}(T \cdot n \cdot k)$ est polynomiale, ce qui établit que l'apprentissage supervisé par descente de gradient est dans **PTIME** et donc Turing-calculable.

7.2 Perspectives

Extension à la classification. On peut remplacer la régression linéaire par un perceptron sigmoïdal $\hat{y} = \sigma(wx+b)$. La phase **FORWARD** calcule alors $\sigma(z) = (1+e^{-z})^{-1}$, qui nécessite une approximation polynomiale sur le ruban.

Réduction à un seul ruban. Implémenter la simulation mono-ruban et mesurer expérimentalement le surcoût quadratique théorique prévu par le Théorème 2.1.

Critère d'arrêt basé sur le gradient. Ajouter un ruban dédié à la norme $\|\nabla\mathcal{L}\|$ et une phase CONVERGE qui arrête la machine lorsque $\|\nabla\mathcal{L}\| < \varepsilon$, encodant ainsi un critère d'arrêt adaptatif.

Réseaux de neurones. La généralisation à un réseau multicouche nécessite une MTM avec un nombre de rubans proportionnel aux couches ($k = 2L + 3$ pour un réseau à L couches), et la mise en œuvre de la rétropropagation comme séquence de transitions δ_k .

Références bibliographiques

- [1] A. M. Turing, *On Computable Numbers, with an Application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society, vol. 42, pp. 230–265, 1936.
- [2] M. Sipser, *Introduction to the Theory of Computation*, 3^e éd., Cengage Learning, 2013.
- [3] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3^e éd., Addison-Wesley, 2006.
- [4] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016. Disponible en ligne : <https://www.deeplearningbook.org>
- [5] C. H. Papadimitriou, *Computational Complexity*, Addison-Wesley, 1994.
- [6] Python Software Foundation, *Module `fractions` — Rational numbers*, Python 3.12 Documentation, 2024. <https://docs.python.org/3/library/fractions.html>

Code Python complet du simulateur

```

1  def run(self) -> dict:
2      """Lance la MTM jusqu'a l'etat HALT. Retourne un
        resume."""
3      _transitions = {
4          'INIT':      self._transition_INIT,
5          'LOAD':      self._transition_LOAD,
6          'FORWARD':   self._transition_FORWARD,
7          'LOSS':      self._transition_LOSS,
8          'GRAD':      self._transition_GRAD,
9          'UPDATE':    self._transition_UPDATE,
10         'CHECK':     self._transition_CHECK,
11     }
12     while self.state != 'HALT':
13         _transitions[self.state]()
14         self.step_count += 1
15     return {
16         'w':          float(self.w),
17         'b':          float(self.b),
18         'loss_final': float(self._loss),
19         'steps':      self.step_count,
20     }

```

Listing A.1. Classe MultiTapeTM – méthode run() principale

```

1  def _transition_GRAD(self):
2      """
3      delta(GRAD, ...) -> UPDATE
4      Calcule grad_w = (2/n) * sum((y_hat - y) * x)
5      grad_b = (2/n) * sum(y_hat - y)
6      Ecrit les gradients sur R2[0] et R2[1].

```

```
7      """
8      grad_w = Fraction(0)
9      grad_b = Fraction(0)
10     coef    = Fraction(2, self.n)           # 2/n (exact)
11     for (x, y, y_hat) in self._pairs:
12         err    = y_hat - y
13         grad_w += coef * err * x
14         grad_b += coef * err
15     self._sum_grad_w = grad_w
16     self._sum_grad_b = grad_b
17     self.tapes[2].write_at(0, grad_w)
18     self.tapes[2].write_at(1, grad_b)
19     self.state = 'UPDATE'
```

Listing A.2. Transition GRAD – calcul exact des gradients

Exemple de sortie du simulateur

Sortie console – Expérience 1

```
=====
MTM 5-Rubans -- Apprentissage Supervise (Regression Lineaire)
n=10 exemples | eta=0.0100 | T=500 iterations
=====

delta(INIT      -> LOAD    ) | step=   0 | Init w=0, b=0, t=0
delta(CHECK     -> LOAD    ) | step=  600 | t=100 < T=500
delta(CHECK     -> LOAD    ) | step=1200 | t=200 < T=500
delta(CHECK     -> LOAD    ) | step=1800 | t=300 < T=500
delta(CHECK     -> LOAD    ) | step=2400 | t=400 < T=500
delta(CHECK     -> HALT    ) | step=3000 | t=500 >= T=500
                                Modele: w=2.019862, b=0.903240

=====

HALT atteint en 3001 transitions.
Modele final (R4) : w=2.019862, b=0.903240

=====

MSE=0.017079 | R2=0.999487
```